Voter Guide App: Product Requirements Document

Version 1.0 · October 27, 2026 · Dr. Ernesto Lee · Working name: Voter Guide

1. Summary

A free, nonpartisan web app that turns one home address into everything a voter needs: where to vote, what's on their ballot, plain-English explanations of ballot measures, candidate info, a personal voting plan, their election office, and every district they live in. It is built on the Google Civic Information API, runs as a static site plus serverless functions on Netlify, and has no database.

Important scope note: the app helps people *get ready to vote*. It does not register voters or cast or record ballots. Those happen only through official state and county systems, which the app links to.

2. Goals and non-goals

Goals

- A voter goes from address to a complete, personal voting picture in under 30 seconds.
- Every fact shown is traceable to its source, with official government data clearly marked.
- Ballot measures are understandable by someone with no legal background.
- Zero backend state: no database, no user accounts, no stored addresses.
- Works well on a phone, since most voters will use it on one.

Non-goals (v1)

- Voter registration, ballot casting, or any official transaction.
- Endorsements, recommendations, or any "who should I vote for" feature.
- User accounts, saved profiles across devices, or server-side push notifications.
- Coverage outside the United States.

3. Users

User	Need	What success looks like
First-time voter	Doesn't know where to go or what's on the ballot	Finds polling place and understands every item before election day
Busy regular voter	Wants to vote early or drop a ballot off quickly	Picks a site and time, adds it to their calendar in one tap
Mail voter	Needs drop box locations and deadlines	Sees the nearest drop boxes with hours on a map
Engaged citizen	Wants to know every district and who represents them	Sees all districts and current officeholders for their address

4. Features (the eight modules)

All eight share one entry point: the voter enters their address once, and every module reads from the same lookup. Priority: **P0** = launch, **P1** = fast follow.

#	Module	Priority	Data source
F1	Where do I vote? finder	P0	`voterinfo`: pollingLocations, earlyVoteSites, dropOffLocations
F2	Personalized sample ballot	P0	`voterinfo`: contests, candidates, ballotPlacement
F3	Ballot measure explainer	P0	`voterinfo`: referendum fields + Claude API summary
F4	Candidate research hub	P1	`voterinfo`: candidates (url, channels, photo, contact)
F5	Voting plan and reminders	P0	F1 data + browser storage + calendar file
F6	Election office directory	P0	`voterinfo`: state[].electionAdministrationBody, local_jurisdiction
F7	District mapper	P1	`divisionsByAddress`
F8	Representatives (partner data)	P1	OCD IDs from F7 + partner API (e.g. BallotReady)

F0. Address entry and election picker (shared)

- Single address field with browser autocomplete; "Use my location" is optional and only fills the field.
- On submit, call `/api/voterinfo`. If `otherElections` is returned, show a "Which election?" picker and re-query with that `electionId`.
- Handle every status code with a plain-English message: `noStreetSegmentFound`, `addressUnparseable`, `electionOver`, `electionUnknown`, `multipleStreetSegmentsFound`. Always link to the local election office as the fallback.
- When no election is live, show a "No upcoming election for this address" screen with the election list from `/api/elections` and the district mapper (F7), which still works.
- The address lives only in the URL query string and the voter's own browser. It is never logged or stored server-side.

F1. Where do I vote? finder

- Map (Leaflet + OpenStreetMap tiles, no key needed) with three marker types: Election Day, Early voting, Drop box.
- List view under the map: name, address, hours, date range, accessibility notes, "Get directions" link (opens the phone's maps app).
- Filter chips by type; sort by distance from the entered address.
- If `mailOnly` is true, lead with drop boxes and a mail-voting explainer.
- Locations without lat/long still appear in the list, just not on the map.

F2. Personalized sample ballot

- Every contest in `ballotPlacement` order, grouped by level: Federal, State, County, City, School board, Special district.
- Candidates in `orderOnBallot` order with party, exactly as the ballot lists them. No reordering, no highlighting.
- "Vote for N" shown per contest. Measures link to F3; candidates link to F4.
- Voter can mark private choices on the sample ballot, saved only in their browser, and print it or save it as a PDF to bring to the polls (check local rules on bringing notes).

F3. Ballot measure explainer

- For each referendum: official title and subtitle, passage threshold, effect of abstaining, and the official pro and con statements, shown verbatim and labeled "Official".
- An AI summary (Claude API, via a Netlify Function) answering four fixed questions: What does it do? What changes if it passes? What stays the same if it fails? Who is affected?
- AI text is clearly labeled "AI-generated summary, may contain errors" with the full official text one tap away.
- Neutrality rules in the system prompt: no recommendation, no loaded language, equal treatment of yes and no, summarize only what the official text says.
- Responses cached at Netlify's CDN for up to 24 hours (see Section 7), so each measure is summarized once, not once per voter.

F4. Candidate research hub

- Card per candidate: photo, party, campaign website, social channels, phone, email, all only if the API provides them.
- Missing fields are hidden, never filled with guesses. No AI-written candidate descriptions in v1.
- Optional "Search news" link that opens a neutral web search for the candidate's name and office.

F5. Voting plan and reminders

- Voter picks a method (Election Day, Early, Mail/drop box), a location from F1, and a day and time window.
- Plan is saved in the browser (localStorage) and summarized on a "My plan" card.
- Reminders without a server: one-tap "Add to calendar" downloads an `.ics` file (and Google Calendar link) with alerts the day before and two hours before.
- Share button uses the phone's native share sheet to send the plan to a friend or family member.
- If the election office's `electionNoticeText` is present, show it as a banner at the top of every screen.

F6. Election office directory

- State and local election office: name, phone, email, hours, physical and mailing address.
- Action buttons for every official URL the API returns: Register, Check registration, Absentee voting, Ballot info, Election rules, Find my polling place.
- Tap-to-call and tap-to-email on phones.

F7. District mapper

- List of every division containing the address from `divisionsByAddress`: congressional, state senate, state house, county, city, school board, special districts.
- Each row shows the friendly name and the OCD ID (collapsible, for power users).
- Works year-round, even when no election data is live.

F8. Representatives (partner data)

- Uses the OCD IDs from F7 to look up current officeholders from a partner, since Google retired its Representatives API on April 30, 2025.
- Recommended partner: BallotReady Officeholders API (commercial; needs a signed agreement and key). Fallback: link out to a free lookup tool.

- Ships behind a feature flag so v1 can launch without it.

5. Architecture (Netlify, no database)

A static single-page app served from Netlify's CDN, plus a handful of Netlify Functions that act as a thin, cached proxy. The functions exist for two reasons: keep API keys off the client, and let Netlify cache responses so repeat lookups don't burn quota.

```
Browser (SPA, localStorage for plan + ballot marks)
   |
   v
Netlify CDN  --static assets-->  /index.html, JS, CSS
   |
   v  /api/*  (netlify.toml redirect)
Netlify Functions (cached proxy, keys in env vars)
   |-- /api/elections      -> Google Civic  GET /elections
   |-- /api/voterinfo      -> Google Civic  GET /voterinfo
   |-- /api/divisions      -> Google Civic  GET /divisionsByAddress
   |-- /api/explain        -> Claude API    (measure summary)
   '-- /api/officeholders  -> Partner API   (F8, feature-flagged)
```

Layer	Choice	Why
Front end	Vite + React (or SvelteKit static), Tailwind	Fast static build, deploys as plain files
Maps	Leaflet + OpenStreetMap	Free, no API key, works well on phones
Server	Netlify Functions (Node/TypeScript)	Hides keys, adds caching, no servers to run
Caching	Netlify CDN via `Cache-Control` / `Netlify-CDN-Cache-Control` headers	Replaces a database for shared, repeatable lookups
Client state	URL query string + localStorage	Shareable links; plan and ballot marks stay on the device
Reminders	.ics file + Google Calendar link	Works without storing anyone's contact info
Config	Netlify environment variables	`GOOGLE_CIVIC_API_KEY`, `ANTHROPIC_API_KEY`, `PARTNER_API_KEY`

Why no database works here

- Google's data is the source of truth and changes often, so storing it would mean storing stale data.
- Per-voter data (plan, ballot marks) belongs on the voter's device, which is also better for privacy.
- Shared, expensive work (AI summaries) is cached by Netlify's CDN instead of a table.

6. API contracts (Netlify Functions)

Endpoint	Input	Output	Cache
`GET /api/elections`	none	Google's election list	1 hour
`GET /api/voterinfo`	`address`, optional `electionId`	Google's response, passed through with status mapped to a friendly message	1 hour, keyed by normalized address + electionId
`GET /api/divisions`	`address`	List of {ocdId, name}	24 hours

Endpoint	Input	Output	Cache
`GET /api/explain`	`electionId`, `districtId`, `title`, `address`	{whatItDoes, ifPasses, ifFails, whoIsAffected}. Function pulls the measure text from cached voterinfo itself.	24 hours; cache key limited to electionId + districtId + title via `Netlify-Vary`, so all voters share one summary
`GET /api/officeholders`	`ocdIds[]`	Partner officeholder records	24 hours (F8 only)

Every function: validates input, sets a short timeout, returns a clean error object, and never logs the address.

7. Google Civic API rules the app must follow

Rule	How the app complies
Don't cache voting locations and contests for more than 24 hours (absent cache headers)	voterinfo cached 1 hour; nothing is stored long-term
Don't cache districts for more than 30 days	divisions cached 24 hours
No A/B tests, holdbacks, or experiments; every user gets the same experience	No experiment framework; one experience for everyone
Extra quota only for user-facing apps; increases take 1-2 months	App is user-facing; file the quota request by end of August before any November election
Elections expire after election day	"No upcoming election" screen + F7 keep the app useful year-round
Data changes in the final days	"Check back before you vote" note on F1 and F2, plus a last-updated timestamp
Attribution is optional but must use Google's exact wording if used	Footer: "Voting information powered by the Civic Information API" linking to Google's page

8. Non-functional requirements

Area	Requirement
Performance	First screen under 2 seconds on a mid-range phone over 4G; address-to-results under 3 seconds when not cached
Accessibility	WCAG 2.1 AA: keyboard navigation, screen-reader labels, 4.5:1 contrast, text resizable to 200%
Mobile	Mobile-first layout; map and list usable one-handed
Privacy	No accounts, no analytics on addresses, no third-party trackers; privacy page states this plainly
Neutrality	No endorsements; ballot order preserved; AI summaries labeled and constrained; same treatment for every party and side
Security	API keys only in Netlify env vars; functions rate-limited per IP; strict Content-Security-Policy
Reliability	If Google is down or out of quota, show the election office (when known) and the state's official lookup link
Language	English at launch; Spanish in v1.1 (strings externalized from day one)

9. Success metrics (measured without tracking people)

- Lookup success rate: share of address lookups returning `success` (counted in function logs, no addresses).

- Calendar adds per lookup: how many voters actually make a plan.
- Explainer cache hit rate: proves the no-database caching is working and keeps AI costs flat.
- Function error rate under 1% in the two weeks before an election.

10. Release plan

Phase	Scope	Exit criteria
v1.0 (MVP)	F0, F1, F2, F3, F5, F6	Works end to end for a live election address in at least 3 states
v1.1	F4, F7, Spanish	Districts work for any US address; Spanish strings reviewed by a native speaker
v1.2	F8 (partner officeholders)	Partner agreement signed; key in env vars; flag turned on

11. Risks and open questions

Risk / question	Mitigation or decision needed
Testing between elections: voterinfo returns little when nothing is live	Use Google's test election ID 2000 for development and record sample responses as fixtures
AI summary gets a measure wrong	Constrained prompt, official text always one tap away, clear AI label, "Report a problem" link
Quota runs out on election week	1-hour CDN caching; file a quota increase well ahead of the election
Partner data costs money (F8)	Decide budget and vendor before v1.2; keep feature flagged off until then
Netlify function timeouts on slow AI calls	Keep the summary prompt short; return official text immediately and load the AI summary after
Open: app name and domain	Decide before launch